Open a pull request

Create a new pull request by comparing changes across two branches. If you need to, you can also compare across forks. Learn more about diff comparisons here.

base: develop ← compare: DATAUSER-5787-Dynamic-Attri...

✓ **Able to merge.** These branches can be automatically merged.

Reviewers

Suggestions

A466449 Request

Assignees

No one—assign yourself

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Use Closing keywords in the description to automatically close issues

1 commit | 9 files changed | 1 contributor

Commits on Apr 6, 2026

feat(attribute): Implement attribute management component in the UI ··· 447f5a3

viknesh-vallinayagam committed 16 minutes ago

Showing 9 changed files with 1,235 additions and 525 deletions.

Split | Unified

5 package.json

```
117  117      "react-router-dom": "^5.2.0",
118  118      "react-toastify": "^6.2.0",
119  119      "reactstrap": "^8.10.1",
120       -  "remove-accents": "^0.5.0"
     120  +  "remove-accents": "^0.5.0",
     121  +  "@jsonforms/core": "3.1.0",
     122  +  "@jsonforms/react": "3.1.0",
     123  +  "@jsonforms/vanilla-renderers": "3.1.0"
121  124    },
122  125    "resolutions": {
123  126      "@types/react": "17.0.2",
```

3 src/sdc-dataset/components/Dataset/locales/en.json

```
906  906    "dataset.dataSourceFile": "Business data file",
907  907    "dataset.dataSourceFile_originalFileName": "Excel (BusinessData)",
908  908    "businessData.card.multipleCommunities": "Multiple communities",
909       - "tooltip.multipleCommunities": "There are multiple communities linked to this business data. It can be changed by deleting the one that is displayed and adding the correct one"
     909  + "tooltip.multipleCommunities": "There are multiple communities linked to this business data. It can be changed by deleting the one that is displayed and adding the correct one",
     910  + "dataset.dataSource.details.tabs.tables.details.tabs.attributes": "Attributes"
910  911  }
```

34

src/sdc-dataset/components/data-source/tabs/tables/table-details/TableDetails.tsx

```
10   10   import { DataSourceTableService } from '../../../../../../services/dataSourceTable.service';
11   11   import { DataSourceTableDetails } from '../../../../../../services/dataSourceTable.service.types';
12   12   import { ServiceError } from '../../../../shared/ServiceError';
     13 + import Attributes from './tabs/attributes/Attributes';
13   14   import AuditTrail from './tabs/audit-trail/AuditTrail';
14   15   import TableSchema from './tabs/table-schema/TableSchema';
15   16
16      - type TableDetailsTab = 'TABLE_SCHEMA' | 'AUDIT_TRAIL';
     17 + type TableDetailsTab = 'TABLE_SCHEMA' | 'AUDIT_TRAIL' | 'ATTRIBUTES';
17   18
18   19   enum TableDetailsTabHash {
19   20     TABLE_SCHEMA = '#schema',
20   21     AUDIT_TRAIL = '#audit-trail',
     22 +   ATTRIBUTES = '#attributes',
```

```diff
21  23   }
22  24
23  25   interface TableDetailsProps {
78  80       };
79  81     }, [configuration, tableId]);
80  82
81     -   // Functions to switch table details tabs
82     -   const goToTableSchemaTab = () => {
83     -     setCurrentTab('TABLE_SCHEMA');
84     -   };
85     -   const goToAuditTrailTab = () => {
86     -     setCurrentTab('AUDIT_TRAIL');
87     -   };
    83 +   const goToTableSchemaTab = () => setCurrentTab('TABLE_SCHEMA');
    84 +   const goToAuditTrailTab = () => setCurrentTab('AUDIT_TRAIL');
    85 +   const goToAttributesTab = () => setCurrentTab('ATTRIBUTES');
88  86
89     -   // ON 'location.hash' CHANGE -> redirect to the right tab
90  87     useEffect(() => {
91  88       if (location && location.hash) {
92  89         switch (location.hash) {
96  93           case TableDetailsTabHash.AUDIT_TRAIL:
97  94             goToAuditTrailTab();
98  95             break;
    96 +         case TableDetailsTabHash.ATTRIBUTES:
    97 +           goToAttributesTab();
    98 +           break;
99  99           default:
100 100            goToTableSchemaTab();
101 101            break;
192 192                      </a>
193 193                    </li>
194 194                    <li className="nav-item">
195    -                     {/* TODO: remove disabled when add audit trail part */}
196 195                      <a
197 196                        className={`nav-link disabled${currentTab ===
    'AUDIT_TRAIL' ? ' active' : ''}`}
198 197                        href={TableDetailsTabHash['AUDIT_TRAIL']}
204 203                        <FormattedMessage
    id="dataset.dataSource.details.tabs.tables.details.tabs.auditTrail" />
205 204                      </a>
206 205                    </li>
    206 +                  <li className="nav-item">
    207 +                    <a
    208 +                      className={`nav-link${currentTab === 'ATTRIBUTES' ? '
    active' : ''}`}
    209 +                      href={TableDetailsTabHash['ATTRIBUTES']}
    210 +                      onClick={(event) => {
    211 +                        event.preventDefault();
    212 +                        goToAttributesTab();
    213 +                      }}
    214 +                    >
```

```
+                        <FormattedMessage id="dataset.dataSource.details.tabs.tables.details.tabs.attributes" />
+                      </a>
+                    </li>
                    </ul>
                    <div className="align-items-start align-self-stretch flex-grow-1">
-                     {/* Table schema part */}
                      {currentTab === 'TABLE_SCHEMA' && <TableSchema tableDetails={tableDetails} />}
-                     {/* Audit trail part */}
                      {currentTab === 'AUDIT_TRAIL' && <AuditTrail />}
+                     {currentTab === 'ATTRIBUTES' && <Attributes tableId={tableId} />}
                    </div>
                  </div>
                </div>
```

263

...c-dataset/components/data-source/tabs/tables/table-details/tabs/attributes/Attributes.t…

```
@@ -0,0 +1,263 @@
+ import React, { MutableRefObject, useCallback, useEffect, useRef, useState } from 'react';
+
+ import { JsonForms } from '@jsonforms/react';
+ import { vanillaCells, vanillaRenderers } from '@jsonforms/vanilla-renderers';
+ import { useMyWidgetConfiguration } from '@sg-widgets/react-core';
+ import { FormattedMessage } from 'react-intl';
+
+ import { Loader } from '../../../../../../../../common/components/shared/Loader';
+ import { MyConfiguration } from '../../../../../../../../common/my-configuration';
+ import { AttributeService } from '../../../../../../../../services/attribute.service';
+ import {
+   AttributeCategory,
+   AttributeEntry,
+   AttributeViewResponse,
+ } from '../../../../../../../../services/attribute.service.types';
+ import { ServiceError } from '../../../../../../../shared/ServiceError';
+ import SgInputControl from './SgInputControl';
+ import SgInputControlTester from './SgInputControlTester';
+
+ interface AttributesProps {
+   tableId: number;
+ }
+
+ const customRenderers = [...vanillaRenderers, { tester: SgInputControlTester, renderer: SgInputControl }];
```

```
+
+ const chunkAttributes = (attributes: AttributeEntry[], size: number): AttributeEntry[][] => {
+   const chunks: AttributeEntry[][] = [];
+   for (let i = 0; i < attributes.length; i += size) {
+     chunks.push(attributes.slice(i, i + size));
+   }
+   return chunks;
+ };
+
+ const buildSingleAttrProps = (attr: AttributeEntry, localValues: Record<string, string>) => {
+   const currentValue =
+     localValues[attr.technicalName] !== undefined
+       ? localValues[attr.technicalName]
+       : attr.value !== undefined && attr.value !== null
+       ? String(attr.value)
+       : '';
+
+   const schema = {
+     type: 'object',
+     properties: {
+       [attr.technicalName]: {
+         type: attr.dataType,
+         title: attr.name,
+       },
+     },
+   };
+   const uischema = {
+     type: 'Control',
+     scope: `#/properties/${attr.technicalName}`,
+   };
+   const data = { [attr.technicalName]: currentValue };
+   return { schema, uischema, data };
+ };
+
+ const Attributes: React.FC<AttributesProps> = ({ tableId }: AttributesProps) => {
+   const configuration: MyConfiguration | undefined = useMyWidgetConfiguration<MyConfiguration>();
+   const abortControllerRef: MutableRefObject<AbortController | undefined> = useRef<AbortController>();
+
+   const [attributeData, setAttributeData] = useState<AttributeViewResponse | undefined>();
+   const [isLoading, setIsLoading] = useState<boolean>(true);
+   const [isError, setIsError] = useState<boolean>(false);
+   const [saveError, setSaveError] = useState<string | undefined>();
+   const [isSaving, setIsSaving] = useState<boolean>(false);
+
+   const [localValues, setLocalValues] = useState<Record<string, Record<string, string>>>({});
```

```
+
+   const [editingCategories, setEditingCategories] = useState<Record<string, boolean>>({});
+
+   const fetchAttributeValues = useCallback(() => {
+     if (!configuration || !tableId) return;
+     abortControllerRef.current = new AbortController();
+     setIsLoading(true);
+     setIsError(false);
+     setSaveError(undefined);
+
+     AttributeService.getAttributeValues(configuration.dataCatalogApiBaseUrl, tableId, abortControllerRef.current.signal)
+       .then((response: AttributeViewResponse | void) => {
+         setAttributeData(response || undefined);
+         setLocalValues({});
+         setEditingCategories({});
+         setIsLoading(false);
+       })
+       .catch(() => {
+         setIsError(true);
+         setIsLoading(false);
+       });
+   }, [configuration, tableId]);
+
+   useEffect(() => {
+     fetchAttributeValues();
+     return () => {
+       if (abortControllerRef.current) {
+         abortControllerRef.current.abort();
+       }
+     };
+   }, [fetchAttributeValues]);
+
+   const handleLocalChange = (category: string, technicalName: string, value: string) => {
+     setLocalValues((prev) => ({
+       ...prev,
+       [category]: {
+         ...(prev[category] || {}),
+         [technicalName]: value,
+       },
+     }));
+   };
+
+   const handleEdit = (category: string) => {
+     setEditingCategories((prev) => ({ ...prev, [category]: true }));
+   };
+
+   const handleCancel = (category: string) => {
+     setLocalValues((prev) => ({ ...prev, [category]: {} }));
+     setEditingCategories((prev) => ({ ...prev, [category]: false }));
```

```
+   };
+
+   const handleSave = (categoryData: AttributeCategory) => {
+     if (!configuration) return;
+     setIsSaving(true);
+     setSaveError(undefined);
+
+     const categoryLocalValues = localValues[categoryData.category] || {};
+     const editableAttrs = categoryData.attributes.filter((a) => !a.readOnly);
+
+     const valuesToSave: Record<string, string> = {};
+     editableAttrs.forEach((attr) => {
+       const edited = categoryLocalValues[attr.technicalName];
+       if (edited !== undefined) {
+         valuesToSave[attr.technicalName] = edited;
+       } else {
+         valuesToSave[attr.technicalName] = attr.value !== null && attr.value !== undefined ? String(attr.value) : '';
+       }
+     });
+
+     AttributeService.upsertAttributeValues(configuration.dataCatalogApiBaseUrl, tableId, {
+       category: categoryData.category,
+       values: valuesToSave,
+     })
+       .then(() => {
+         setIsSaving(false);
+         setEditingCategories((prev) => ({ ...prev, [categoryData.category]: false }));
+         fetchAttributeValues();
+       })
+       .catch(() => {
+         setIsSaving(false);
+         setSaveError('Failed to save. Please try again.');
+       });
+   };
+
+   const categoryHasEditableField = (categoryData: AttributeCategory): boolean =>
+     categoryData.attributes.some((attr) => !attr.readOnly);
+
+   if (isLoading) {
+     return <Loader divContainerClass="d-flex justify-content-center w-100 mt-4" loadingSpanClass="display-5" />;
+   }
+
+   if (isError) {
+     return <ServiceError section="attribute values" />;
+   }
+
```

```
+   if (!attributeData || attributeData.categories.length === 0) {
+     return (
+       <div className="d-flex flex-column align-items-center justify-content-center p-5 text-secondary">
+         <i className="icon icon-md mb-2">info</i>
+         <span>
+           <FormattedMessage id="dataset.dataSource.details.tabs.tables.details.tabs.attributes.empty" />
+         </span>
+       </div>
+     );
+   }
+
+   return (
+     <div style={{ overflowY: 'auto', maxHeight: 'calc(100vh - 320px)', padding: '1.5rem' }}>
+       {saveError && (
+         <div className="alert alert-danger mb-3" role="alert">
+           {saveError}
+         </div>
+       )}
+       {attributeData.categories.map((categoryData: AttributeCategory) => {
+         const rows = chunkAttributes(categoryData.attributes, 3);
+         const hasEditable = categoryHasEditableField(categoryData);
+         const isEditing = !!editingCategories[categoryData.category];
+         const catLocalValues = localValues[categoryData.category] || {};
+
+         return (
+           <div key={categoryData.category} style={{ marginBottom: '2rem', marginTop: '1rem' }}>
+             <div className="d-flex align-items-center justify-content-between mb-3">
+               <h5 className="mb-0" style={{ fontWeight: 600 }}>
+                 {categoryData.category}
+               </h5>
+               {hasEditable && (
+                 <div className="d-flex gap-2">
+                   {isEditing ? (
+                     <>
+                       <button
+                         type="button"
+                         className="btn btn-md btn-flat-secondary"
+                         disabled={isSaving}
+                         onClick={() => handleCancel(categoryData.category)}
+                       >
+                         Cancel
+                       </button>
+                       <button
+                         type="button"
+                         className="btn btn-md btn-primary"
+                         disabled={isSaving}
+                         onClick={() => handleSave(categoryData)}
```

```
+                        >
+                          {isSaving ? 'Saving...' : 'Save'}
+                        </button>
+                      </>
+                    ) : (
+                      <button
+                        type="button"
+                        className="btn btn-md btn-outline-primary btn-icon-text"
+                        onClick={() => handleEdit(categoryData.category)}
+                      >
+                        <i className="icon">edit</i>
+                        Edit
+                      </button>
+                    )}
+                  </div>
+                )}
+              </div>
+              <div className="card card-bordered p-4">
+                {rows.map((rowAttrs: AttributeEntry[], rowIndex: number) => (
+                  <div key={rowIndex} className="row mb-2">
+                    {rowAttrs.map((attr: AttributeEntry) => {
+                      const isFieldReadOnly = attr.readOnly || !isEditing;
+                      const { schema, uischema, data: formData } = buildSingleAttrProps(attr, catLocalValues);
+                      return (
+                        <div key={attr.attributeId} className="col-4">
+                          <JsonForms
+                            schema={schema}
+                            uischema={uischema}
+                            data={formData}
+                            renderers={customRenderers}
+                            cells={vanillaCells}
+                            readonly={isFieldReadOnly}
+                            onChange={() => {}}
+                            config={{
+                              onLocalChange: (path: string, value: string) =>
+                                handleLocalChange(categoryData.category, attr.technicalName, value),
+                            }}
+                          />
+                        </div>
+                      );
+                    })}
+                  </div>
+                ))}
+              </div>
+            </div>
+          );
+        })}
+      </div>
+    );
+ };
```

```
+
+ export default Attributes;
```

36

...taset/components/data-source/tabs/tables/table-details/tabs/attributes/SgInputControl.t…

```
@@ -0,0 +1,36 @@
+ import React from 'react';
+
+ import { ControlProps } from '@jsonforms/core';
+ import { withJsonFormsControlProps } from '@jsonforms/react';
+
+ const SgInputControl = ({ data, label, visible, enabled, path, config }: ControlProps) => {
+   if (!visible) {
+     return null;
+   }
+
+   const isReadOnly = !enabled;
+
+   const handleChange = (e: React.ChangeEvent<HTMLInputElement>) => {
+     if (!isReadOnly && config?.onLocalChange) {
+       config.onLocalChange(path, e.target.value);
+     }
+   };
+
+   return (
+     <div className="d-flex flex-column mb-3">
+       <label className="text-secondary font-weight-medium mb-1">{label}</label>
+       <input
+         type="text"
+         className="form-control"
+         value={data !== undefined && data !== null ? String(data) : ''}
+         onChange={handleChange}
+         onKeyDown={isReadOnly ? (e) => e.preventDefault() : undefined}
+         onPaste={isReadOnly ? (e) => e.preventDefault() : undefined}
+         onCut={isReadOnly ? (e) => e.preventDefault() : undefined}
+         style={{ cursor: isReadOnly ? 'default' : 'text' }}
+       />
+     </div>
+   );
+ };
+
+ export default withJsonFormsControlProps(SgInputControl);
```

5

.../components/data-source/tabs/tables/table-details/tabs/attributes/SgInputControlTester.…

```
@@ -0,0 +1,5 @@
+ import { isControl, rankWith } from '@jsonforms/core';
+
+ const SgInputControlTester = rankWith(5, isControl);
+
+ export default SgInputControlTester;
```

36 src/services/attribute.service.ts

```
@@ -0,0 +1,36 @@
+ import { CanceledError } from 'axios';
+
+ import { DXError } from '../common/errors';
+ import { getAxios } from '../common/utils';
+ import { AttributeViewResponse } from './attribute.service.types';
+
+ export class AttributeService {
+   static async getAttributeValues(
+     dataCatalogApiBaseUrl: string,
+     tableId: number,
+     abortSignal?: AbortSignal
+   ): Promise<AttributeViewResponse | void> {
+     try {
+       const url = `${dataCatalogApiBaseUrl}/data-source-tables/${tableId}/attributes`;
+       const response = await getAxios('GET', url, { signal: abortSignal });
+       return response.data as AttributeViewResponse;
+     } catch (err) {
+       if (!(err instanceof CanceledError)) {
+         throw DXError.fromAxios('Failed to get attribute values', err);
+       }
+     }
+   }
+
+   static async upsertAttributeValues(
+     dataCatalogApiBaseUrl: string,
+     tableId: number,
+     payload: { category: string; values: Record<string, unknown> }
+   ): Promise<void> {
+     try {
+       const url = `${dataCatalogApiBaseUrl}/data-source-tables/${tableId}/attributes`;
+       await getAxios('POST', url, { data: payload });
+     } catch (err) {
+       throw DXError.fromAxios('Failed to save attribute value', err);
+     }
+   }
+ }
```

19 src/services/attribute.service.types.ts

```diff
@@ -0,0 +1,19 @@
+ export interface AttributeEntry {
+   attributeId: number;
+   name: string;
+   technicalName: string;
+   dataType: 'string' | 'integer' | 'number' | 'boolean';
+   readOnly: boolean;
+   value: string | number | boolean | null;
+ }
+
+ export interface AttributeCategory {
+   category: string;
+   attributes: AttributeEntry[];
+ }
+
+ export interface AttributeViewResponse {
+   objectType: string;
+   objectId: number;
+   categories: AttributeCategory[];
+ }
```

1,359 yarn.lock

Load diff

Large diffs are not rendered by default.